UCS2611 Internet Programming Lab
Mini Project Report

PayPact

Group Expense Splitting & Settlement Application

Done by:
Meghna Manimaran - 3122235001079
Rahul Malaikani - 3122235001103

Computer Science and Engineering
Sri Sivesubramaniya Nadar College of Enginnering
Kalavakkum - 603110

Abstract

PayPact is a full-stack web application that simplifies the process of tracking, splitting, and settling shared expenses within groups of people. Targeting use cases such as shared housing, trips, team outings, and similar social settings, PayPact allows users to form expense groups, log expenses with automatic equal-splitting logic, and settle their debts using a real-time payment gateway (Razorpay/UPI).

The system is built on a Django REST Framework backend secured with JWT authentication, and a modern React + Vite frontend styled with Tailwind CSS.

A greedy debt-minimization algorithm computes the optimal set of transactions required to settle all balances, minimizing the number of payments each group member must make. The application exposes a fully RESTful API and persists data using SQLite (development) with provisions to migrate to a production-grade database.

Introduction

Managing shared expenses is a common challenge faced by friends, flatmates, and colleagues. Without a dedicated tool, groups often rely on informal methods that are error-prone and hard to track. Applications like Splitwise popularized automated expense-splitting, but building a self-hosted, customizable alternative allows teams full control over data and integrations.

Problem Statement

The problem PayPact solves can be summarized as follows:

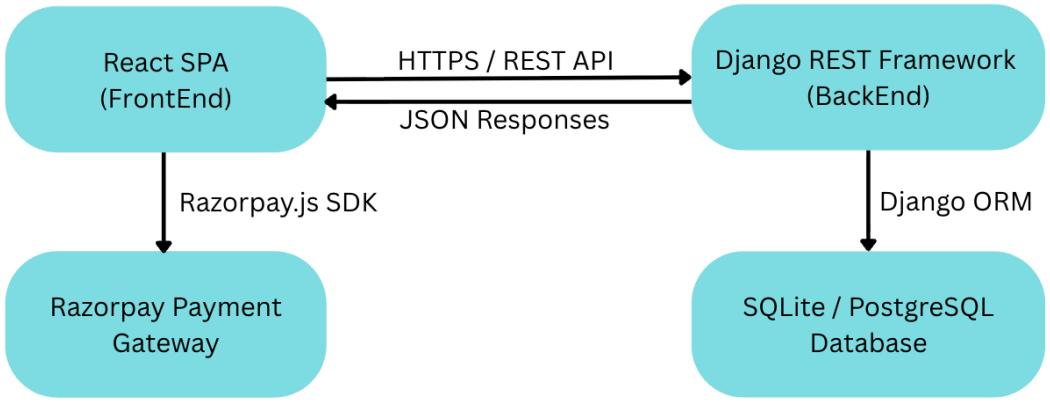
- Multiple people within a group incur shared expenses over time.
- At settlement time, the number of individual transfers needed can be as large as $n(n-1)/2$, where n is the number of group members.
- The system should compute the minimum number of transactions to settle all debts and allow members to pay directly within the app.

Objectives

- Allow user registration and login with secure JWT-based authentication.
- Enable the creation and management of expense groups with specific members.
- Record expenses and automatically calculate per-member balances.
- Compute optimal settlement transactions using a greedy creditor-debtor matching algorithm.
- Integrate with the Razorpay payment gateway so users can pay each other directly within the app.
- Maintain payment audit trails with Razorpay order IDs and signatures.

System Overview

PayPact operates as a two-tier client-server application. The React SPA communicates with the Django REST Framework backend exclusively via JSON over HTTP. The frontend integrates the Razorpay.js SDK for payment processing, while the backend uses Django ORM to interact with the database.



Key Flows

- **Authentication -** User registers/logs in; Backend returns JWT access + refresh tokens, Frontend stores in localStorage and attaches Bearer header to all API calls
- **Group Management** - Authenticated user creates a group, adds other registered users by username, system creates GroupMember entries
- **Expense Logging** - A group member logs an expense (amount, description, who paid) which is stored in Expense table
- **Settlement** - Frontend fetches all expenses + members, computes individual balances. Greedy algorithm identifies who owes whom and displays minimal-transaction settlement plan
- **Payment** - Debtor clicks "Pay Now". Backend creates a Razorpay order. Razorpay modal opens in browser. On success, backend verifies and marks Split as Paid

Technology Stack

Backend Technologies

Technology	Role
Python 3.10+	Core programming language
Django 4.2	Web framework providing ORM, admin, and routing
Django REST Framework	REST API layer with serializers and viewsets
djangorestframework-simplejwt	JWT authentication with access and refresh tokens
django-cors-headers	CORS policy management for frontend communication
Razorpay Python SDK	Payment gateway integration for order and payment management
SQLite	Lightweight embedded database for development

Frontend Technologies

Technology	Role
React 19	Component-based UI framework
Vite 7	Fast build tool and development server
React Router DOM v7	Client-side routing and navigation
Axios	HTTP client with interceptors for authenticated API calls
Tailwind CSS v4 + DaisyUI	Utility-first styling and pre-built component library
Framer Motion	Smooth animations and page transitions
React Hot Toast	Non-intrusive notification toasts
Firebase	Available for auth or future feature extensions

System Architecture & Design

Architectural Pattern

PayPact follows a RESTful Service-Oriented Architecture:

- The backend is stateless, all state is carried in JWT tokens and persisted in the database.
- The frontend is a Single Page Application (SPA), routing is handled client-side by React Router.
- Communication between frontend and backend is exclusively via JSON over HTTP.

Backend Application Structure

```
backend/
├── backend/
│   ├── settings.py        # DB, JWT, CORS, Razorpay keys
│   ├── urls.py            # Root URL configuration
│   ├── asgi.py
│   └── wsgi.py
├── core/
│   ├── models.py          # User, Group, GroupMember, Expense,
Split, Payment
│   ├── views.py           # API views (class-based, DRF APIView /
CreateAPIView)
│   ├── serializers.py     # DRF serializers for all models
│   ├── urls.py            # App-level URL routing
│   ├── permissions.py     # Custom permission: IsMember
│   ├── admin.py
│   └── migrations/
├── manage.py
└── db.sqlite3
```

Frontend Application Structure

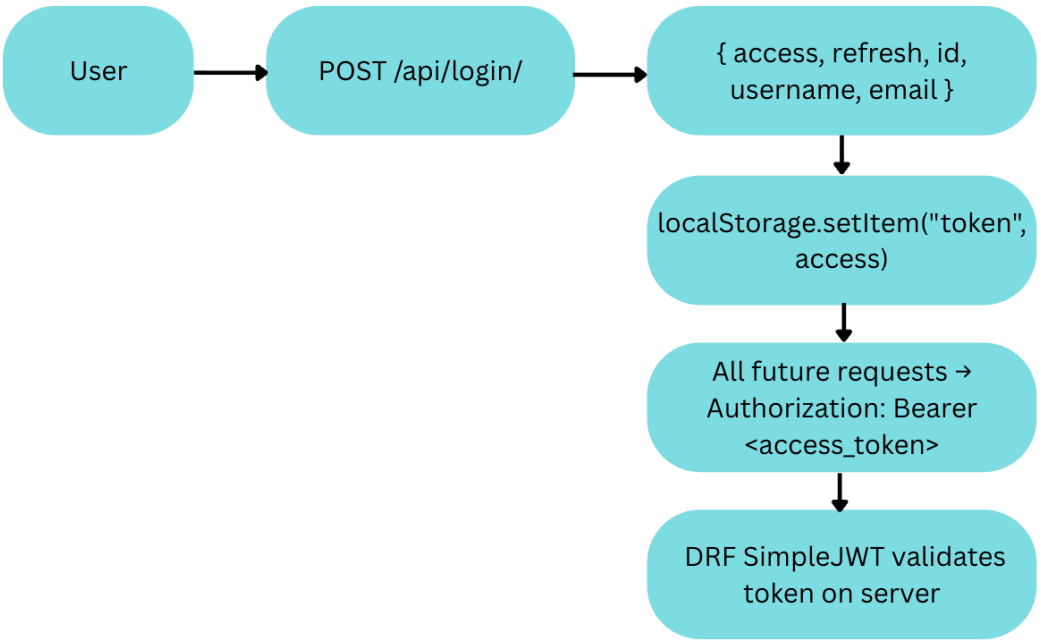
```
frontend/
├── index.html
├── vite.config.js
├── package.json
└── src/
    ├── main.jsx               # React DOM root, BrowserRouter
    ├── App.jsx                # Route definitions
    ├── index.css
    ├── api/
    │   └── axios.js           # Axios instance with JWT
interceptor
    └── pages/
        ├── HomePage.jsx
        ├── LoginPage.jsx
        ├── RegisterPage.jsx
        ├── Dashboard.jsx
        ├── CreateGroupPage.jsx
        ├── GroupPage.jsx
        ├── AddExpense.jsx
        ├── GroupSummary.jsx
        └── Navbar.jsx
```

API Design

All endpoints are prefixed with /api/. Protected endpoints require the Authorization: Bearer <JWT> header.

Method	Endpoint	Auth	Description
POST	/api/register/	Public	Register a new user
POST	/api/login/	Public	Login and receive JWT tokens
POST	/api/create-group/	Required	Create a new expense group
GET	/api/groups/<user_id>/	Required	Get all groups for a user
POST	/api/add-member/	Required	Add a user to a group
POST	/api/add-expense/	Required + Member	Add an expense to a group
GET	/api/expenses/<group_id>/	Required + Member	Get all expenses in a group
GET	/api/users/<username>/	Required	Look up a user by username
GET	/api/group/<pk>/	Required + Member	Get group detail
GET	/api/groups/<group_id>/members/	Required + Member	List group members
GET	/api/group/<group_id>/analytics/	Required + Member	Get expense analytics
POST	/api/createpaymentorder/	Required	Create a Razorpay order
POST	/api/verifypayment/	Required	Verify Razorpay payment
POST	/api/updatesplitstatus/	Required	Mark a split as Paid
GET	/api/groups/<group_id>/splits/	Required	Get all split statuses

Authentication Flow (JWT)



The flow in brief:

1. User submits credentials to POST /api/login/
2. Backend returns - access, refresh, id, username, email
3. Frontend stores the access token in localStorage
4. All subsequent requests include Authorization: Bearer <access_token>
5. DRF SimpleJWT validates the token on every protected endpoint

Implementation Details

1. User Registration & Login

Backend (RegisterUserView, LoginUserView): Registration validates uniqueness of username and email before calling User.objects.create_user(), which hashes the password. Login uses authenticate() to verify credentials, then creates a RefreshToken and returns the access and refresh JWT along with the user's id, username, and email.

Frontend (RegisterPage.jsx, LoginPage.jsx): Simple controlled forms that POST to /api/register/ and /api/login/ respectively. On successful login, the JWT token and user object are stored in localStorage. React state is updated and the user is navigated to /dashboard.

2. Group & Member Management

Backend: CreateGroupView (extends CreateAPIView) accepts name and created_by from the authenticated user. AddMemberToGroupView uses a username/group lookup to prevent duplicate membership via .filter(...).exists(). UserGroupsView queries the GroupMember junction table to return only groups the requesting user belongs to.

Frontend (CreateGroupPage.jsx, GroupPage.jsx): CreateGroupPage allows the user to name the group, then add members by searching usernames via GET /api/users/<username>/. The creator is automatically added as the first member. GroupPage displays the group's expenses and members with quick-access buttons.

3. Expense Recording

Backend (AddExpenseView): Protected by both IsAuthenticated and IsMember permissions. Uses ExpenseSerializer which automatically validates the payload and saves the expense linked to the group.

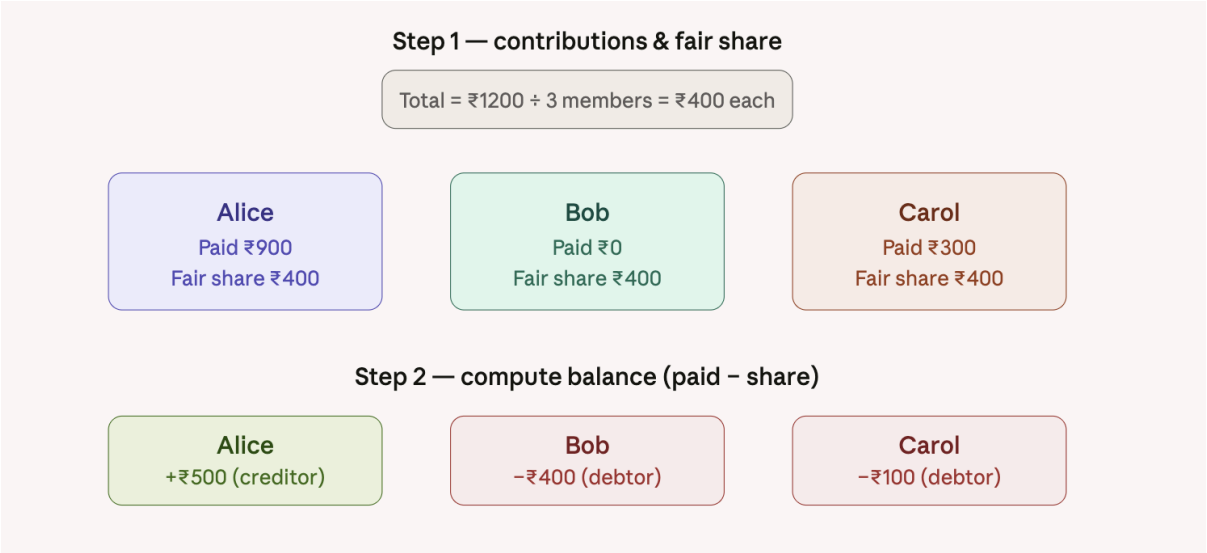
Frontend (AddExpense.jsx): Form captures description, amount, and which member paid. POSTs to /api/add-expense/ and navigates back to the group page on success.

Settlement Algorithm - Greedy Debt Minimization

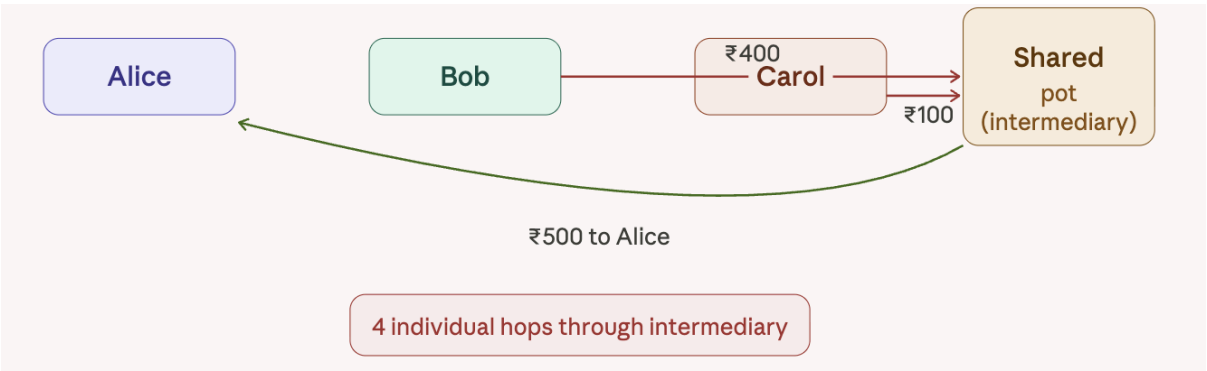
The settlement logic is implemented client-side in GroupSummary.jsx. The steps are:

1. Fetch all expenses and members for the group.
2. Sum total amount paid by each member (membersPaid map).
3. Compute each member's fair share = totalAmount / memberCount.
4. Compute balance for each member = amountPaid − fairShare. A positive balance means the member is a creditor (owed money); a negative balance means they are a debtor (owe money).
5. Sort debtors descending by debt, creditors descending by credit.
6. Greedy two-pointer matching: settled = min(|debtor amount|, creditor amount). Record the transaction, reduce both balances, and advance the pointer when a balance reaches zero.
7. All resulting transactions are displayed with "Pay Now" or "Paid" status.

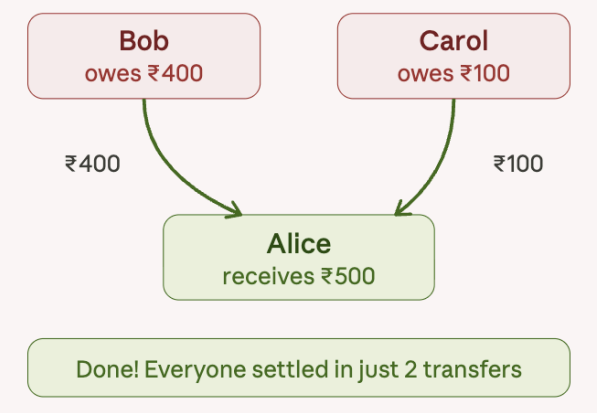
This greedy algorithm minimizes the total number of transactions to at most n−1.



Without Algorithm - Naive approach (4 transactions)



With Algorithm - Greedy Debt Minimization (2 transactions)



Payment Integration (Razorpay)

Backend (CreatePaymentOrderView, VerifyPaymentView, UpdateSplitStatusView):

1. **Order Creation:** Converts the amount to paise (1 INR = 100 paise), creates a Razorpay order via the SDK, saves a Payment record with status 'created', and creates or retrieves the corresponding Split record.
2. **Verification:** On payment completion, updates the Payment record's razorpay_payment_id and razorpay_signature, and sets status to 'success'.
3. **Split Update:** Updates the Split.status field to 'Paid' so the frontend displays the correct settled state.

Frontend (GroupSummary.jsx): Uses the Razorpay.js browser SDK loaded via CDN. Opens the Razorpay payment modal with UPI preferred. The modal's handler function (called on success) sequentially calls /api/verifypayment/ and /api/updatesplitstatus/, then refreshes the summary.

Database Design

PayPact uses Django's ORM with SQLite for development. The schema consists of 6 tables.

Table Definitions

core_user

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
username	VARCHAR(150)	UNIQUE, NOT NULL	Login username
email	VARCHAR(254)	UNIQUE	User email
password	VARCHAR(128)	NOT NULL	Hashed password (PBKDF2)
is_active	BOOLEAN	NOT NULL	Account active flag
date_joined	DATETIME	NOT NULL	Registration timestamp

core_group

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
name	VARCHAR(100)	NOT NULL	Group name
created_by_id	INTEGER	FK → core_user	Creator of the group
created_at	DATETIME	NOT NULL	Auto-set on creation

core_groupmember

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
group_id	INTEGER	FK → core_group	Associated group
user_id	INTEGER	FK → core_user	Member user

core_expense

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
group_id	INTEGER	FK → core_group	Group the expense belongs to
description	VARCHAR(255)	NOT NULL	What the expense was for
amount	DECIMAL(10,2)	NOT NULL	Amount spent
paid_by_id	INTEGER	FK → core_user	Who paid for this expense
created_at	DATETIME	NOT NULL	Auto-set on creation

core_split

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
group_id	INTEGER	FK → core_group	Associated group
owed_by_id	INTEGER	FK → core_user	The user who owes money
owed_to_id	INTEGER	FK → core_user	The user who is owed money
amount	DECIMAL(10,2)	NOT NULL	Settlement amount
status	VARCHAR(20)	DEFAULT 'Unpaid'	Unpaid or Paid

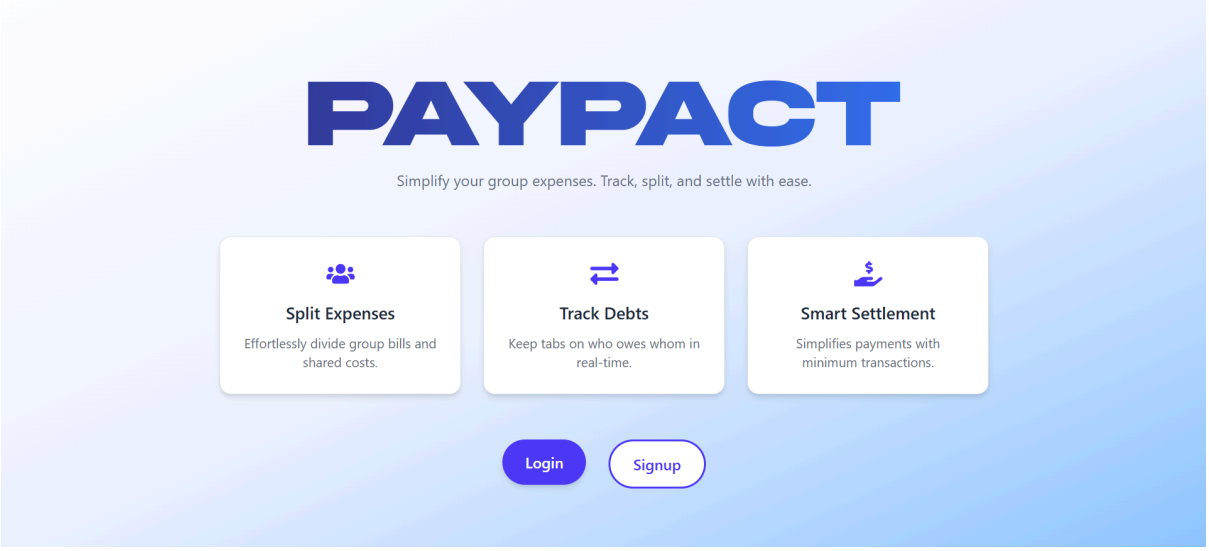
core_payment

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Primary key
group_id	INTEGER	FK → core_group	Associated group
payer_id	INTEGER	FK → core_user	Who made the payment
payee_id	INTEGER	FK → core_user	Who received the payment
amount	DECIMAL(10,2)	NOT NULL	Amount paid
razorpay_payment_id	VARCHAR(100)	NULLABLE	Razorpay payment ID
razorpay_order_id	VARCHAR(100)	NULLABLE	Razorpay order ID
razorpay_signature	VARCHAR(255)	NULLABLE	Payment verification signature
status	VARCHAR(20)	DEFAULT 'initiated'	initiated / created / success / failed
created_at	DATETIME	NOT NULL	Auto-set on creation

Output Screenshots:

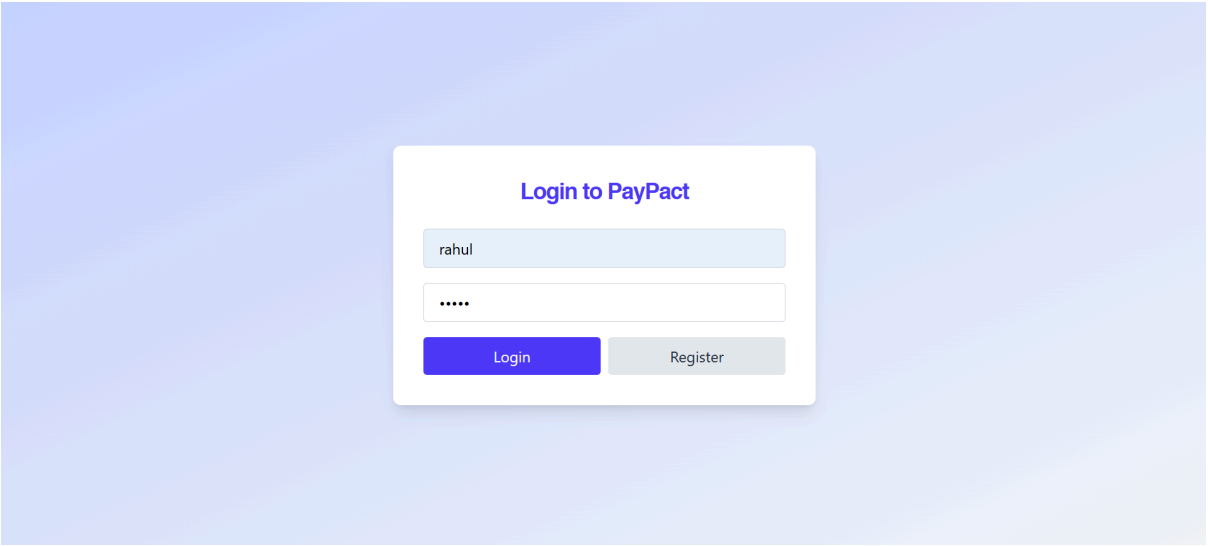
Home Page (/)

A landing/marketing page introducing PayPact. Includes a hero section with a call-to-action to sign up or log in. Built with TailwindCSS gradient backgrounds and Framer Motion entry animations.



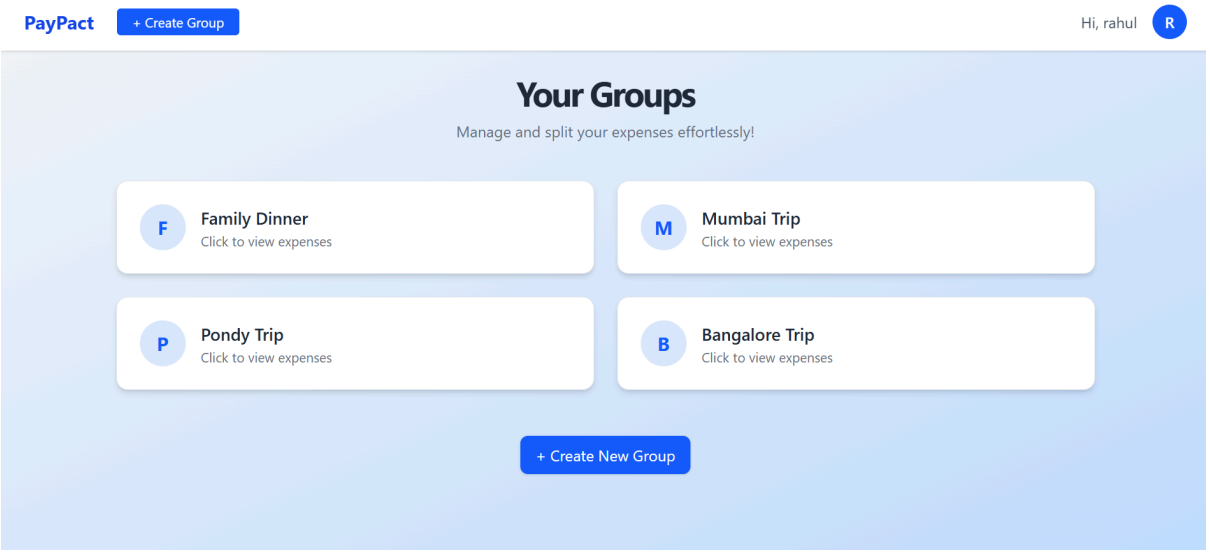
Login Page (/login)

A form with username and password fields. On success, stores token and user in localStorage, then redirects to /dashboard.



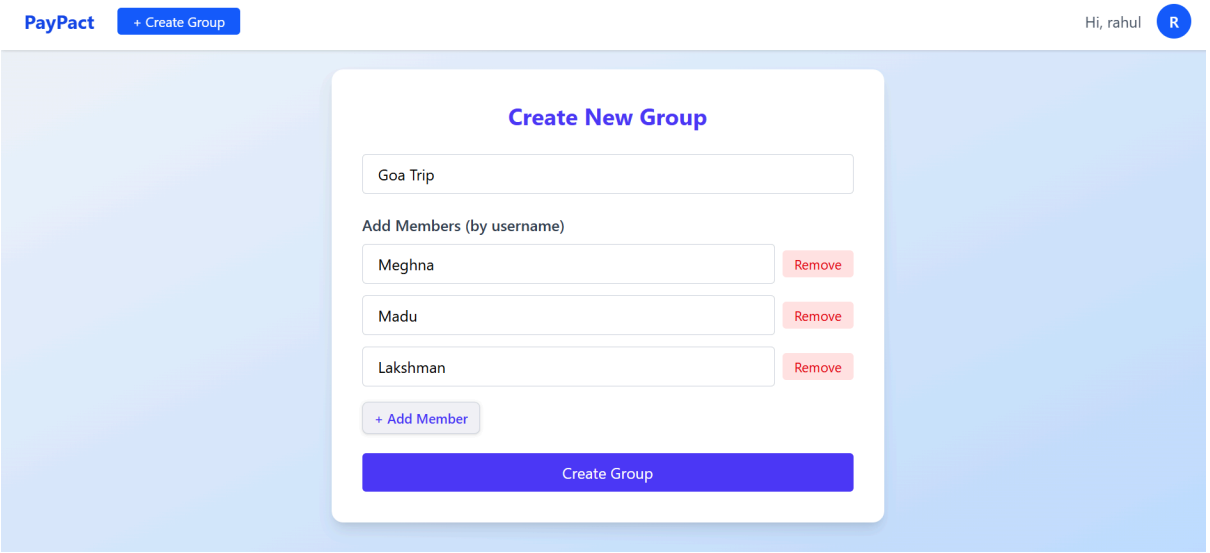
Dashboard (/dashboard)

Displays all groups the logged-in user belongs to, fetched from GET /api/groups/<user_id>/. Each group card shows the group name and creation date, alongside a prominent "Create New Group" button. Non-authenticated users are redirected to /login.



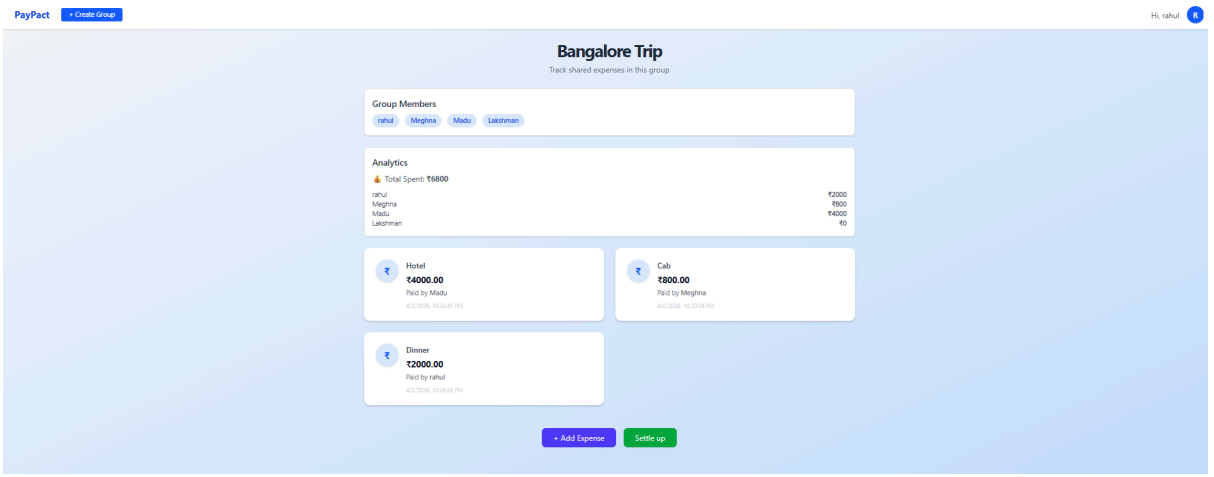
Create Group Page (/create-group)

A multi-step UI where the user: (1) enters a group name, (2) searches for and adds members by username, and (3) submits to create the group and add all members in sequence.



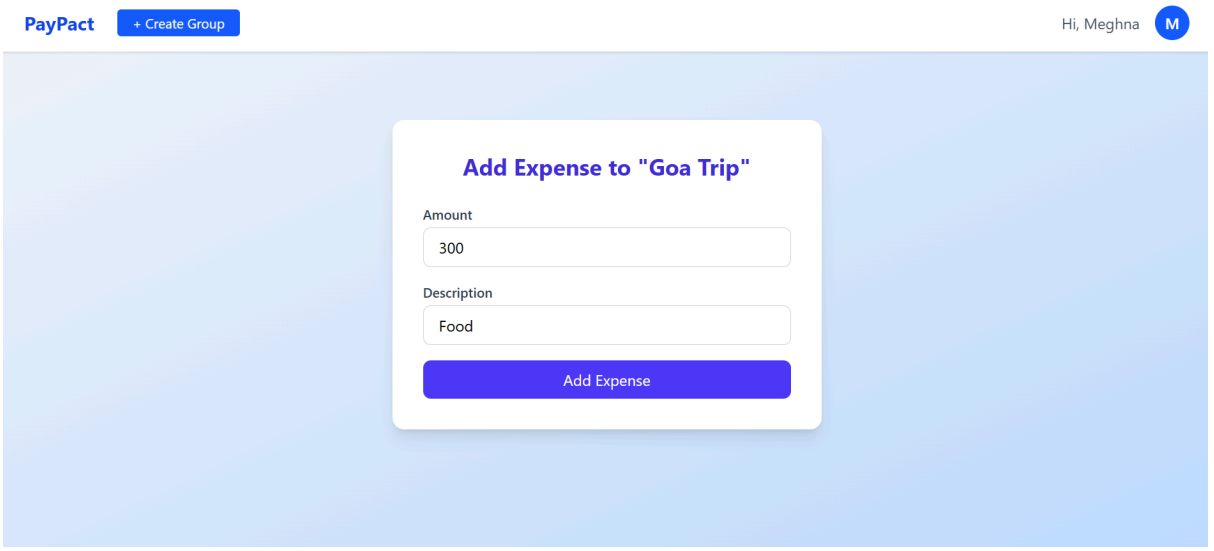
Group Page (/group/:id)

Shows a specific group's name, member list (with option to add more members), a chronological list of all logged expenses, and action buttons for "Add Expense" and "View Summary."



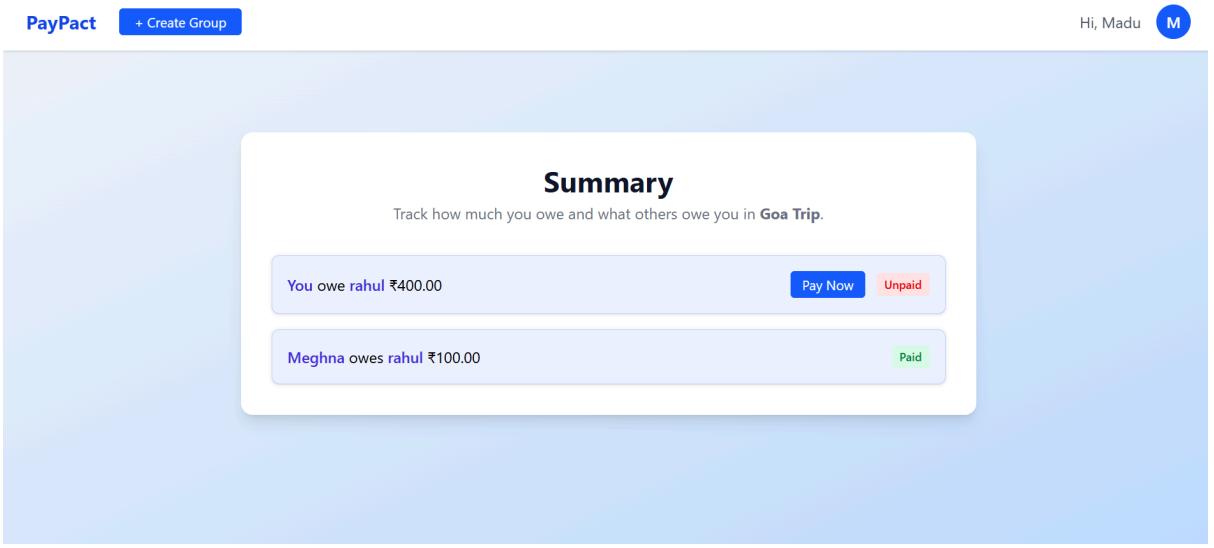
Add Expense Page (/group/:id/add-expense)

A form capturing description, amount, and a "Paid By" dropdown populated with group members. Submits via POST to /api/add-expense/.



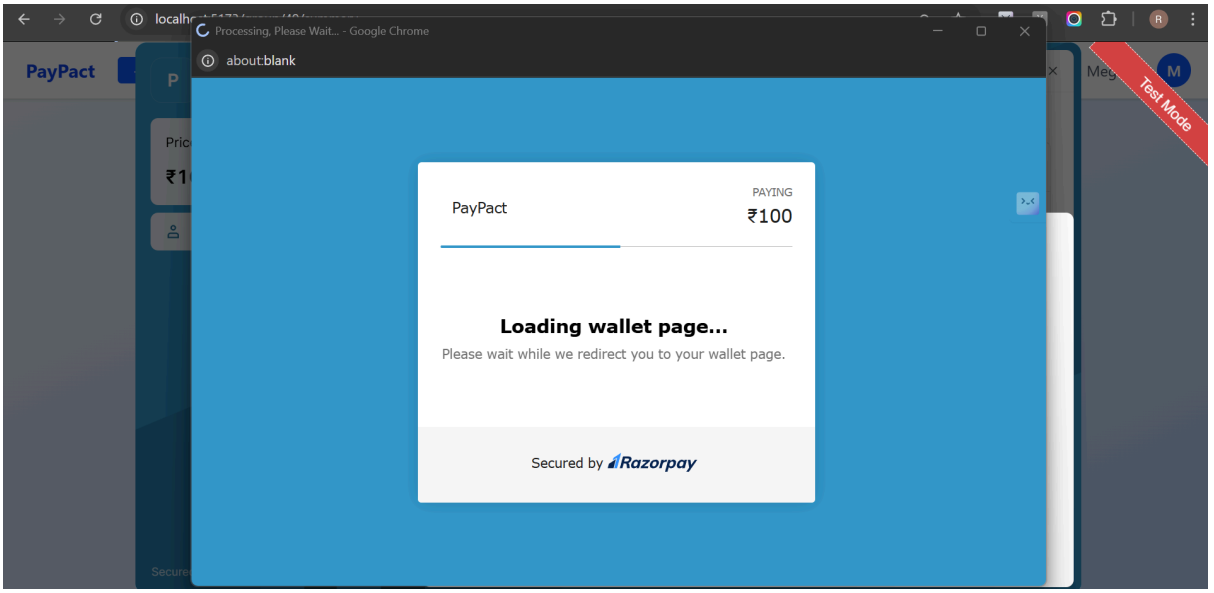
Group Summary Page (/group/:id/summary)

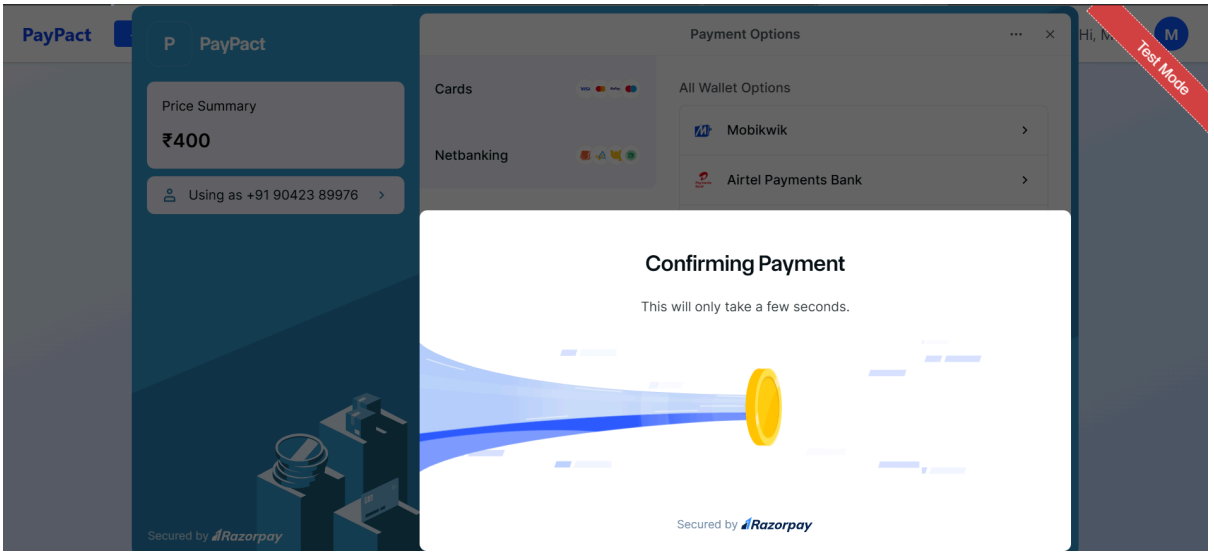
The most feature-rich screen. Displays the minimized settlement plan calculated by the greedy algorithm. Each row shows "[Debtor] owes [Creditor] ₹[Amount]." If the logged-in user is the debtor, a "Pay Now" button triggers the Razorpay modal. After successful payment, the row shows a green "Paid" badge. When all balances are cleared, the screen displays "Everyone is settled up!"

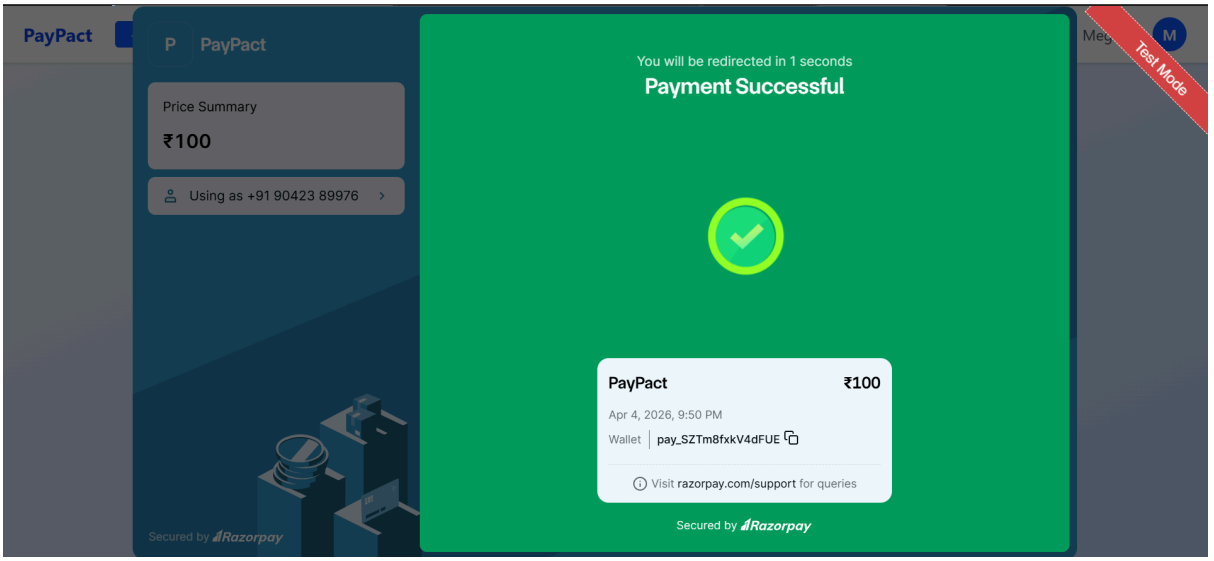


Navbar (Navbar.jsx)

A shared navigation component rendered on all authenticated pages. Shows the PayPact brand, a link to Dashboard, and a Logout button that clears localStorage and redirects to /login.







Result

Algorithm Efficiency - The greedy settlement algorithm has a time complexity of O(n log n) due to sorting, and produces at most n−1 transactions for n group members. This is optimal for the standard minimum-transactions formulation where all members must be individually settled.

Security Considerations

- Passwords are stored as PBKDF2 SHA256 hashes (Django default).
- All protected endpoints require a valid JWT access token.
- The IsMember permission enforces group-level data isolation — users cannot view or interact with groups they are not members of.
- Razorpay payment signatures should be cryptographically verified in production using HMAC-SHA256 (currently skipped in the test flow).

Future Improvements

- **Equal split only** - Support custom/percentage splits
- **No push notifications** - Add WebSocket or Firebase Cloud Messaging
- **No multi-currency** - Integrate currency conversion API
- **Signature verification skipped** - Implement full Razorpay HMAC-SHA256 verification
- **SQLite in development** - Migrate to PostgreSQL for production
- **No profile pictures** - Add file upload support (S3 or Django media)
- **No email notifications** - Add Django email backend for settlement reminders
- **Firebase auth not wired** - Complete OAuth flow with Google sign-in

Conclusion

PayPact successfully delivers a functional, full-stack group expense management and settlement platform. Built entirely on open-source technologies (Django REST Framework and React/Vite) it demonstrates a clean separation of concerns between the backend API layer and the frontend SPA.

The core innovation is the client-side greedy debt minimization algorithm that transforms an arbitrary set of group expenses into the smallest possible set of settlement transactions. The integration of Razorpay's payment gateway further elevates the application from a mere tracking tool to an end-to-end settlement platform where debts can be paid within the same interface.

The project establishes a solid architectural foundation that can be extended with custom split ratios, notification systems, email summaries, mobile apps, and multi-currency support as future enhancements. Its modular design, clear model definitions, DRF serializers, class-based views, and React page components makes it straightforward for new contributors to onboard and extend.
